Lecture 1

DLD Prerequisites and Processor Architecture

Overview

This document covers the foundational number system conversions required for Assembly Language, followed by the architecture of a basic microcomputer. It traces the path from the System Clock through the Instruction Execution Cycle and ends with Pipelining as a method for improving processor throughput.

Base Conversions

Why This Matters

Assembly Language and machine code are expressed in binary, but humans read and write in decimal, octal, and hexadecimal for convenience. The correct conversion method depends on the base: decimal requires the weightage method, while octal and hexadecimal allow a faster direct substitution because they are both powers of 2.

Decimal to Binary (Weightage Method)

Convert base-10 to base-2 by breaking the number into powers of 2. This is the only base among the three that *requires* this subtraction process, since decimal is not a power of 2.

Step-by-Step: $(45)_{10}$

1. Write out the binary weights (powers of 2) from right to left, down to 1: `32 16 8 4 2 1`.
2. Does 32 fit into 45? Yes. Write `1` under 32. Subtract: 45 −32 =13.
3. Does 16 fit into 13? No. Write `0` under 16.
4. Does 8 fit into 13? Yes. Write `1` under 8. Subtract: 13 −8 =5.
5. Does 4 fit into 5? Yes. Write `1` under 4. Subtract: 5 −4 =1.
6. Does 2 fit into 1? No. Write `0` under 2.
7. Does 1 fit into 1? Yes. Write `1` under 1. Subtract: 1 −1 =0.
8. **Result:** Combine the bits in order: $(101101)_2$.

$(256)_{10}=(100000000)_2$

since 256 $=2^8$ exactly, this one resolves in a single step.

Octal to Binary (Direct Substitution)

Convert each base-8 digit directly into a **3-bit** binary group.

Step-by-Step: $(76)_8$

1. Treat 7 and 6 as two different number to perform weightage method on and solve 7:
2. Write out the binary weights (powers of 2) from right to left, down to 1: `4 2 1`.
3. Does 4 fit into 7? Yes. Write `1` under 4. Subtract: $7-4=3$.
4. Does 2 fit into 3? Yes. Write `1` under 2. Subtract: $3-2=1$.
5. Does 1 fit into 1? Yes. Write `1` under 1. Subtract: $1-1=0$.
6. **Result:** Combine the bits in order: $7=(111)$
7. Move to 6 after finding 7.
8. Does 4 fit into 6? Yes. Write `1` under 4. Subtract: $6-4=2$.
9. Does 2 fit into 2? Yes. Write `1` under 2. Subtract: $2-2=0$.
10. Does 0 fit into 1? No. Write `0` under 1.
11. Combine both to find $(76)_8=(111110)_2$

Hexadecimal to Binary (Direct Substitution)

Convert each base-16 digit directly into a **4-bit** binary group.

Step-by-Step: $(A7)_{16}$

1. Treat A and 7 as two different numbers to perform weightage method on
2. Solve A first and since its written as hexadecimal treat it as 10.
3. Write out the binary weights (powers of 2) from right to left, down to 1: `8 4 2 1`.
4. Does 8 fit into 10? Yes. Write `1` under 8. Subtract: $10-8=2$.
5. Does 4 fit into 2? No. Write `0` under 2.
6. Does 2 fit into 2? Yes. Write `1` under 2. Subtract: $2-2=0$.
7. Does 1 fit into 0? No. Write `0` under 1.
8. **Result:** Combine the bits in order: $A=(1010)$
9. Move to 7 after finding A.
10. Does 8 fit into 7? No. Write `0` under 8.
11. Does 4 fit into 7? Yes. Write `1` under 4. Subtract: $7-4=3$.
12. Does 2 fit into 3? Write `1` under 2. Subtract: $3-2=1$.
13. Does 1 fit into 1? Write `1` under 1. Subtract: $1-1=0$.

14. Combine both to find $(A7)_{16} = (10100111)_2$

Basic Microcomputer Design

A processor's architecture relies on specialized internal components that communicate with external hardware over buses.

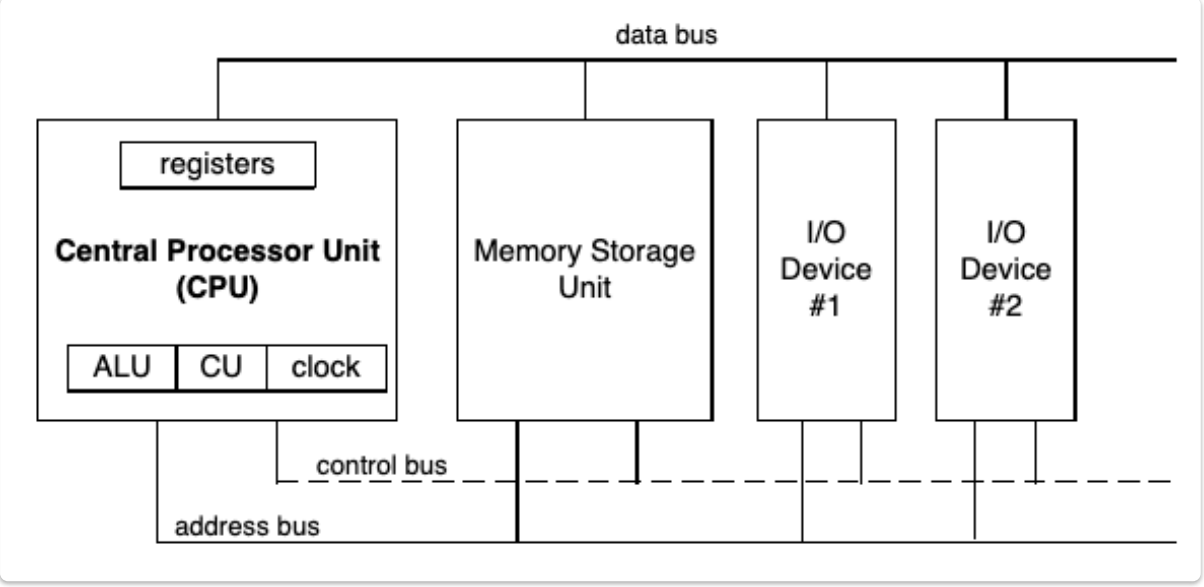
Core CPU Components

- Control Unit (CU): Coordinates the sequence of all execution steps.
- Arithmetic Logic Unit (ALU): Performs mathematical arithmetic and bitwise logical processing.

System Buses

System buses are the physical pathways that move data across the motherboard.

Bus Type	Function
Data Bus	Moves actual data between the CPU, Main Memory (RAM), and I/O Devices.
Address Bus	Transmits the address generated by the CPU to locate specific data.
Control Bus	Carries commands from the CPU indicating whether data should be read or written.



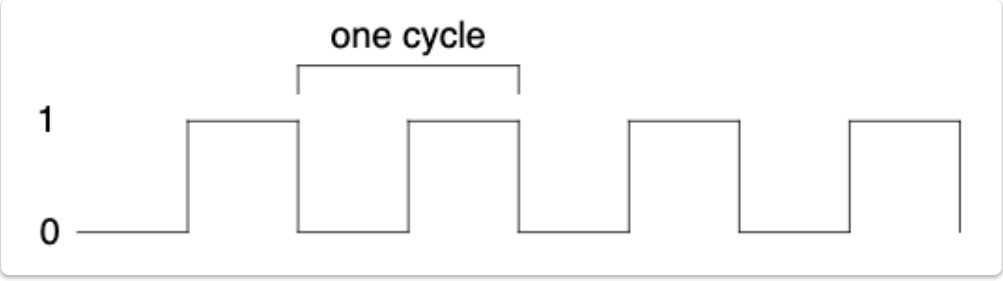
The System Clock

The System Clock serves as the metronome for the entire architecture.

- **Function**: It synchronizes all CPU and BUS operations. Clock pulses are explicitly used to trigger hardware events.
- **Machine Cycle**: A clock cycle measures the precise time required to complete a single operation.
- **Frequency**: The frequency (f) is mathematically defined as:

$$f = 1t$$

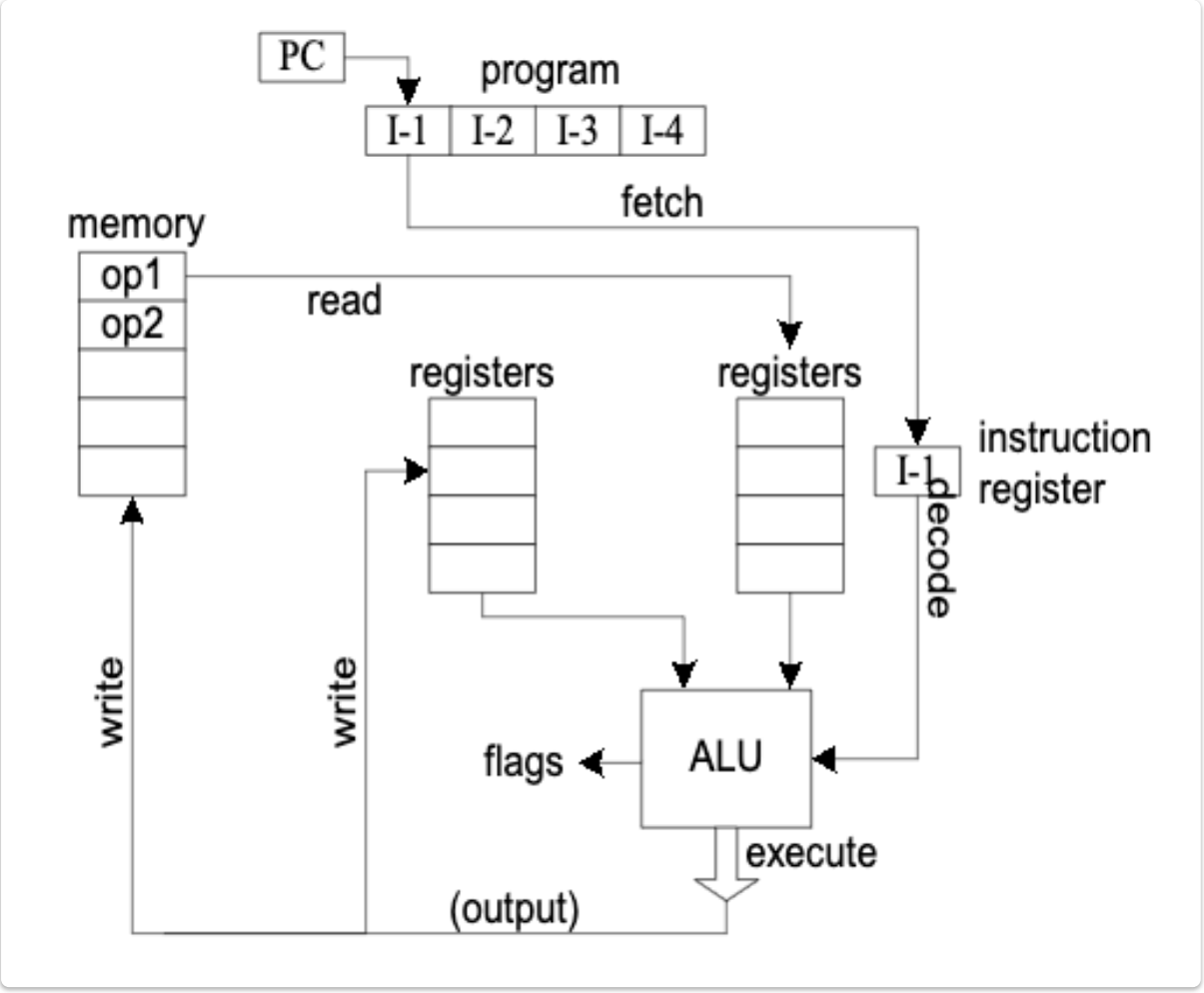
where t is the duration of one cycle.



The Instruction Execution Cycle

The Instruction Pointer / Program Counter (IP/PC) manages the CPU's stage sequence by storing the memory address of the next instruction. The CPU repeatedly loops through five stages:

1. **Fetch**: The CPU pulls the upcoming instruction from memory and loads it into the Instruction Register (IR).
2. **Decode**: The Control Unit decodes the instruction to determine the operation to perform.
3. **Fetch Operands**: The CPU retrieves any required data variables (operands) needed for the calculation.
4. **Execute**: The ALU processes the math or logic task.
5. **Store Output / Write**: The final result is written back to a register or memory location.

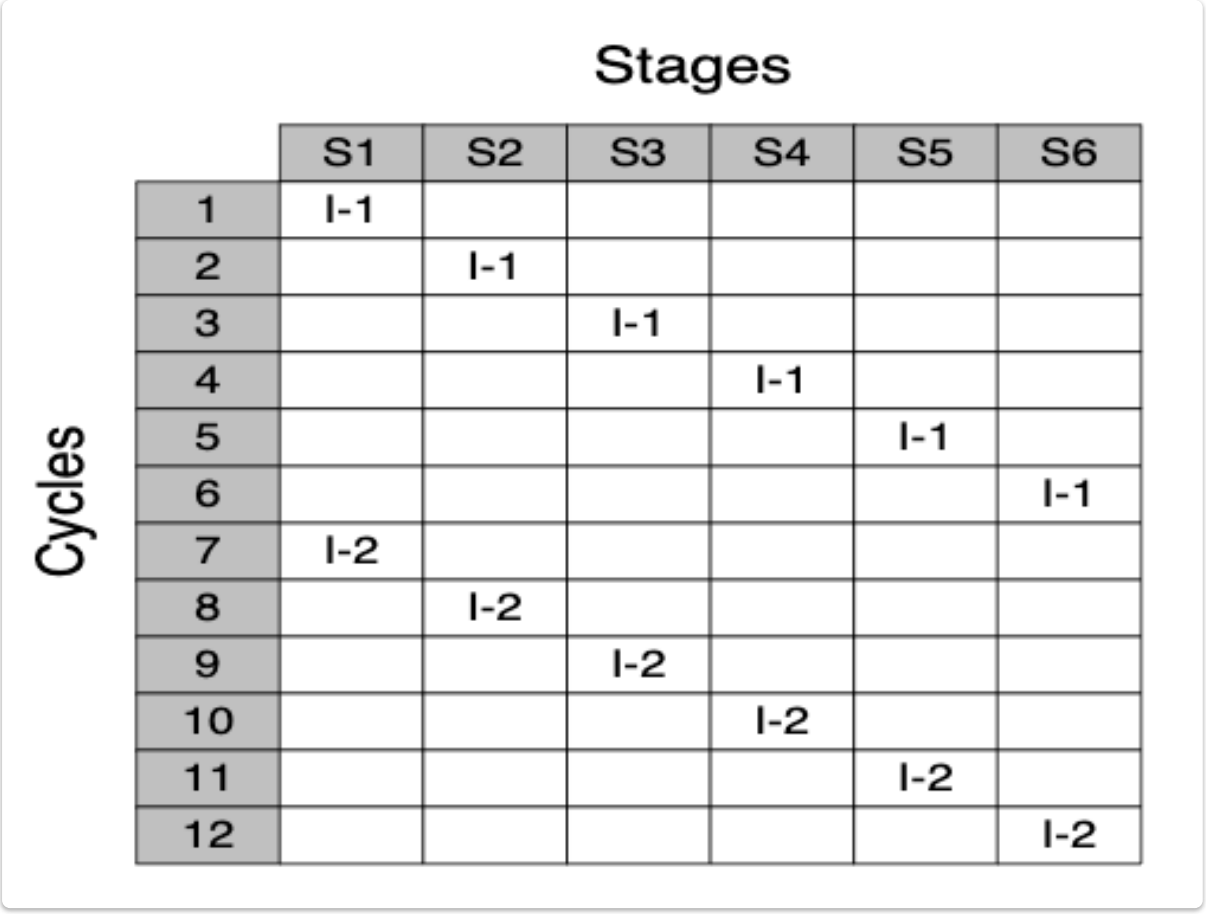


Pipelining and Processor Efficiency

Execution is divided into discrete stages. How a CPU handles these stages determines its throughput.

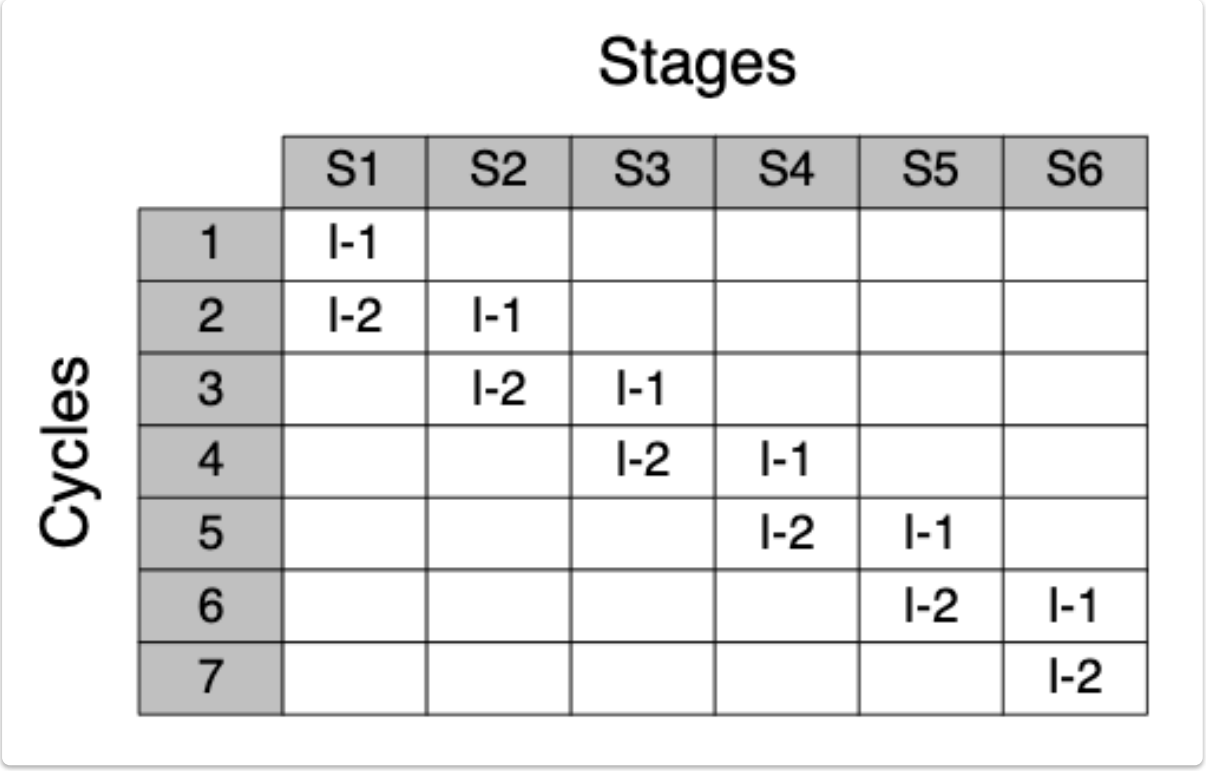
Non-Pipelined Processors

Process one instruction fully from *Fetch* to *Store* before starting the next. This sequential approach results in many wasted cycles where hardware sits idle.



Multi-Stage Pipeline

Makes it possible for the processor to execute instructions in parallel by overlapping the execution stages. As one instruction enters *Decode*, the next enters *Fetch*. This creates far more efficient use of cycles and greater overall throughput.



Pipeline Cycle Formula

$k+(n-1)$

- k = number of pipeline stages
- n = number of instructions

This gives the *total cycles*, not the cycles per instruction. The first instruction still takes k cycles to fully complete — pipelining hides latency for later instructions, it does not eliminate the latency of any single one.

Worked Example: For a 5-stage pipeline executing 2 instructions:

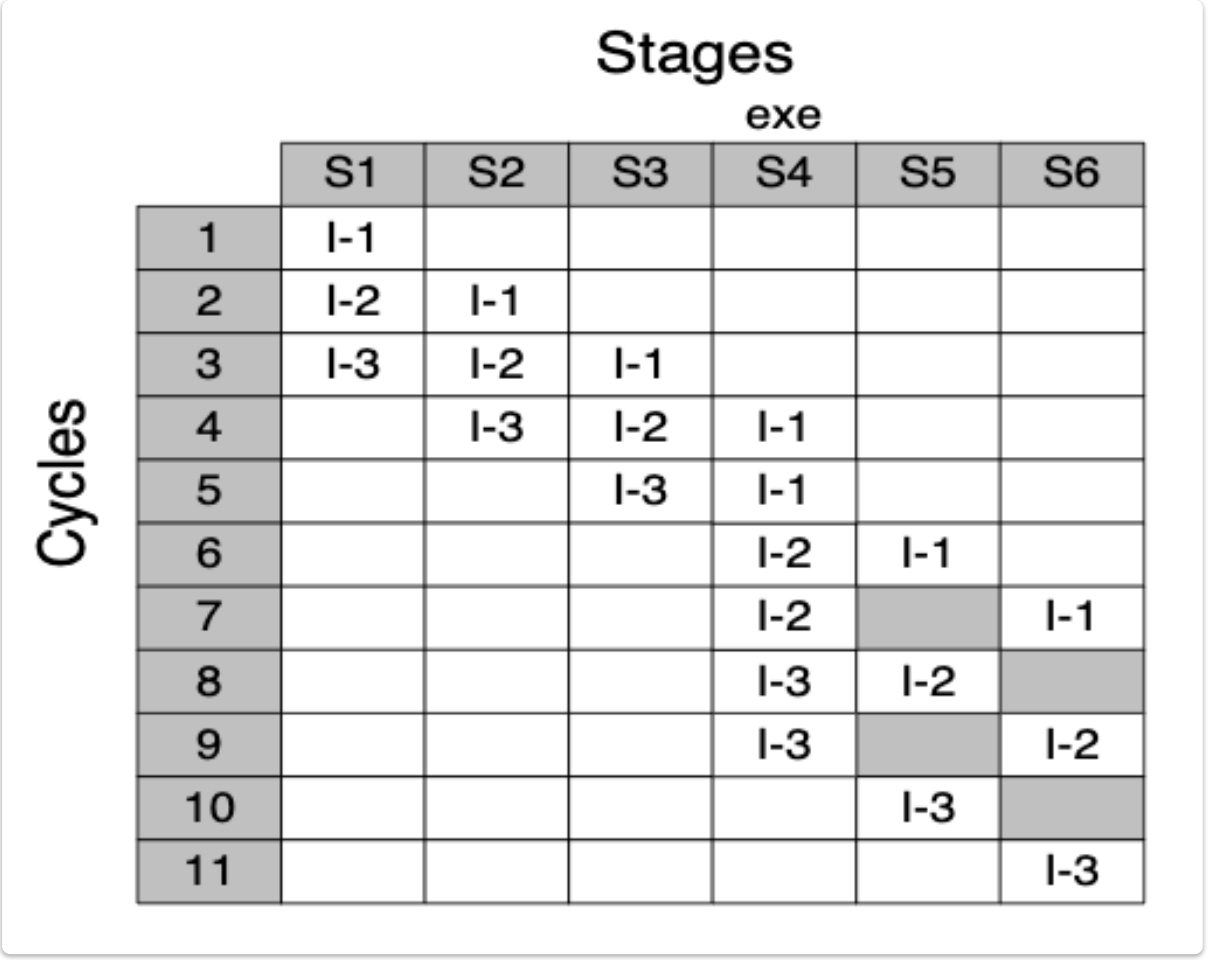
$5+(2-1)=6$ total cycles

Pipelining and Wasted Cycles

Processors can use varying pipeline models. For example, a **6-stage model** may include: Fetch, Decode, Register Read, Fetch Operand, Execute, and Store/Write. Even with a well-designed pipeline, delays still happen.

Wasted Cycles

Clock cycles are wasted when one pipeline stage requires two or more cycles to complete a complex or long instruction. This breaks the "one instruction per cycle" ideal that pipelining aims for.



☰ Extended-Instruction Formula

When *every* instruction requires an extra cycle in the bottleneck stage, the required cycle count becomes:

$k+(2n-1)$

- k = number of pipeline stages
- n = number of instructions

Worked Example: For a 5-stage pipeline (k =5) processing 3 instructions (n =3):

$5+(2(3)-1)=10$ cycles

Caveat: this assumes uniform stalling across all instructions. If only some instructions are long, calculate the added stall cycles individually instead of doubling n across the board.

Lecture 2

IA-32 Architecture, Cache, and Memory Interfacing

Overview

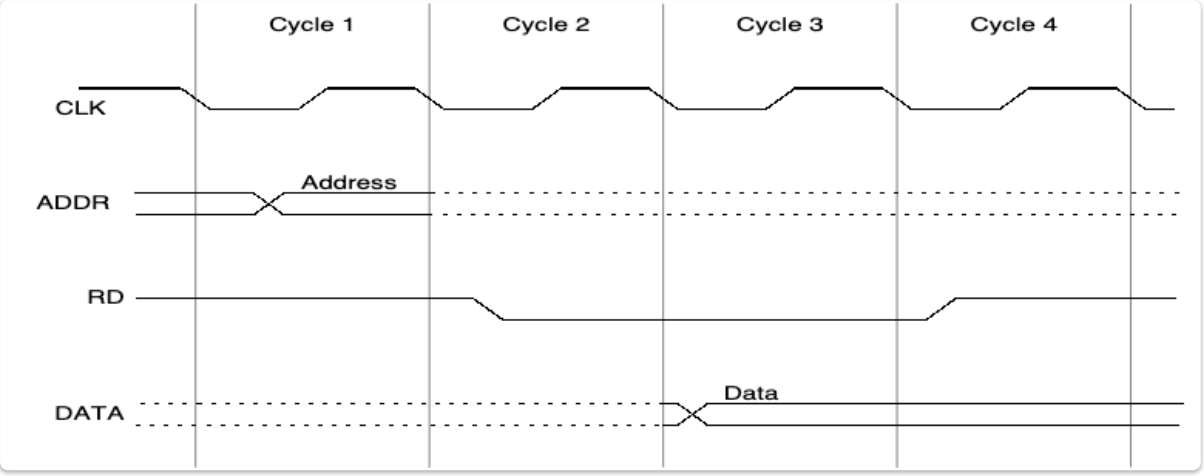
This document covers Memory Interfacing timing, the cache memory hierarchy, IA-32 operating modes, and the full set of CPU registers that make up the basic execution environment.

Memory Interfacing

Memory responds far more slowly than the CPU, so multiple machine cycles are required to read a single piece of data.

Memory Read Sequence

1. The target address is placed on the Address Bus.
2. The Read Line (RD) signal is set low (0).
3. The CPU waits one cycle for memory to respond.
4. The Read Line returns to high (1), indicating the data is now successfully on the Data Bus.



Cache Memory

Cache is high-speed, expensive static RAM used to speed up data access between the CPU and main memory.

Level	Location	Purpose
L1 Cache	Physically inside the CPU	Fastest access, smallest capacity
L2 Cache	Located outside the CPU	Larger, slightly slower than L1

- Cache Hit: Occurs when the data the CPU needs is already sitting in the cache.
- Cache Miss: Occurs when the needed data is *not* in the cache, forcing a slower fetch from main memory.

IA-32 Processor Architecture and Multitasking

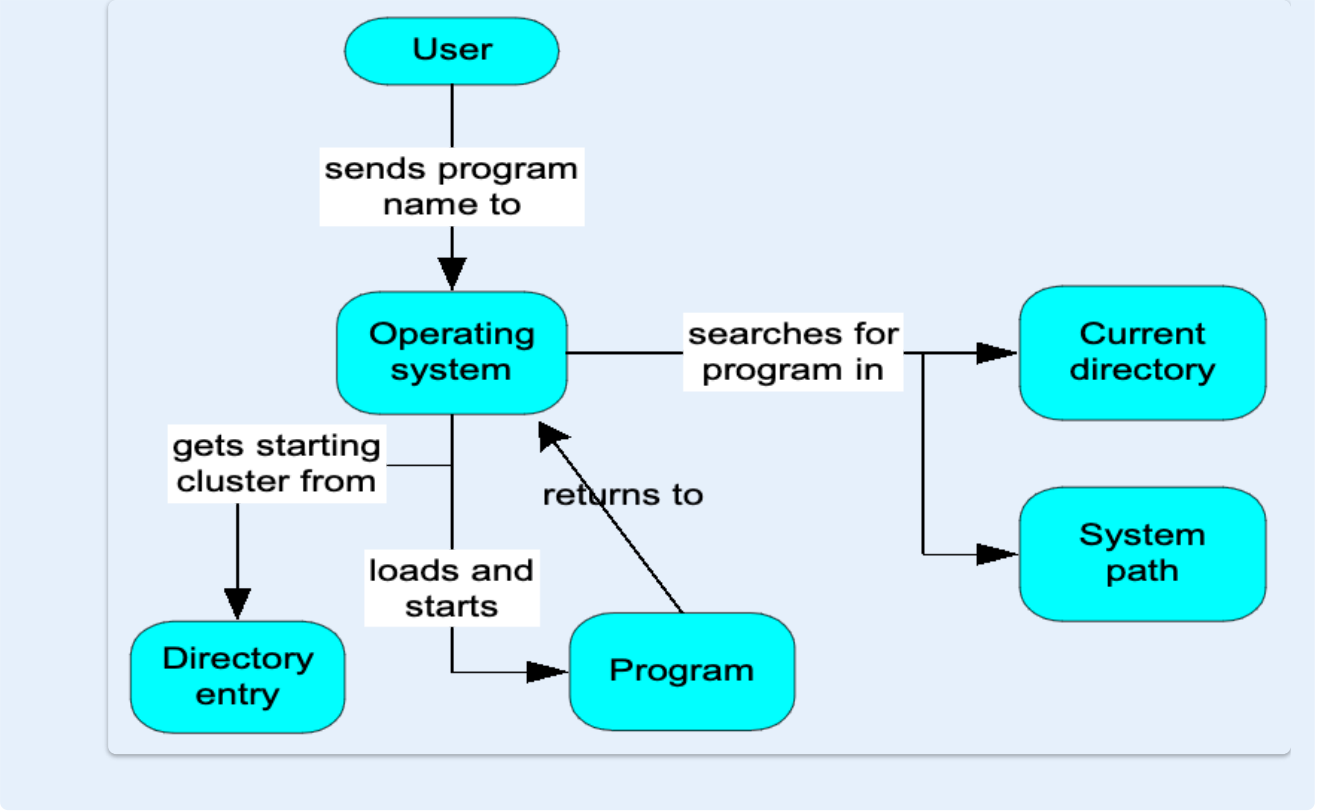
IA-32 is the 32-bit x86 processor architecture.

Multitasking

The OS runs multiple programs or threads at the same time using a scheduler utility that assigns CPU time to each program, rapidly switching tasks to create the illusion of simultaneous execution.

Step-by-Step Flow

1. Initiation: The user begins the process by sending the program name to the Operating System.
2. Searching: The OS searches for the requested program file in two primary locations:
   - The Current Directory
   - The System Path
3. Locating Data: Once the OS finds the program, it retrieves the starting cluster information directly from the file's Directory Entry.
4. Execution: Using that starting cluster, the OS loads the program into memory and starts its execution.
5. Termination: After the program finishes running, it returns control back to the Operating System.



Modes of Operation

Mode	Description
Protected Mode	The native mode for modern OSs like Windows and Linux. Supports 4 GB of addressable RAM using 32-bit addresses, with each program assigned a protected memory partition.
Real-Address Mode	The native mode for old MS-DOS. Mimics the 8086 processor, offering only 1 MB of address space via 20-bit addresses.
System Management Mode	Used for power management, security, and diagnostics.
Virtual-8086 Mode	A hybrid of protected mode where each program runs inside its own isolated, safe 8086 computer environment.

Registers and Basic Execution Environment

Registers are named, high-speed storage locations located directly inside the CPU.

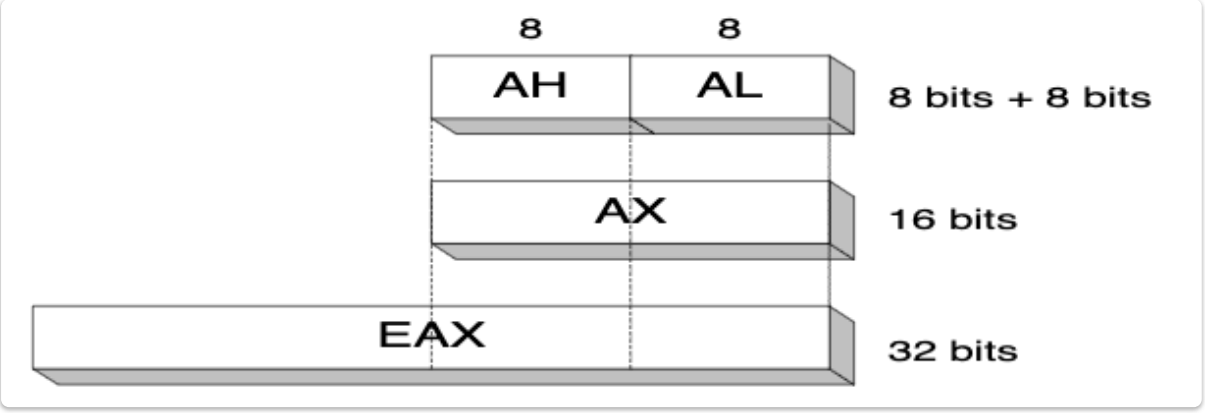
General-Purpose Registers

Register	Full Form
EAX	*Extended Accumulator Register*

Register	Full Form
EBX	*Extended Base Register*
ECX	*Extended Counter Register*
EDX	*Extended Data Register*

Register Sizes

You can access different portions of these registers using specific sub-names. For example, the 32-bit EAX can be accessed as the 16-bit AX, which further splits into two 8-bit halves: AH (high) and AL (low).



Lecture 3

IA-32 Registers, EFLAGS, and Protected Mode

> **Overview**
>
> This note details the full set of IA-32 registers, walks through worked binary arithmetic examples for each EFLAGS status flag, and reviews the memory model used in Protected Mode.

General-Purpose Registers

High-speed storage locations inside the CPU used for arithmetic, data movement, and logical operations.

Register	Role	Notes
EAX	*Accumulator*	Used heavily for arithmetic operations, mathematical logic, and handling return values from function calls.
EBX	*Base Register*	Commonly used as a base pointer for memory access and array indexing.
ECX	*Loop Counter*	Automatically decremented by loop instructions to control loop iterations.
EDX	*Data Register*	Used alongside EAX for complex arithmetic (multiplication and division) and for I/O port access.

Index and Pointer Registers

These hold memory addresses to navigate data structures and the stack.

Register	Role	Notes
ESP	*Stack Pointer*	Always points to the current top of the stack.
EBP	*Base Pointer*	Serves as a fixed reference point for accessing local variables and parameters on the stack during function calls. Often called a "frame pointer" for this reason, though its formal name is Base Pointer.

Register	Role	Notes
ESI / EDI	*Index Registers*	Typically used as source (ESI) and destination (EDI) pointers for high-speed memory block transfers and string operations.

Specialized and Segment Registers

- EIP (Instruction Pointer): Holds the exact memory address of the next instruction the CPU needs to execute.
- Segment Registers (CS, DS, SS, ES, FS, GS): Hold pointers to different memory segments (Code, Data, Stack, and extra segments) to organize a program's structure.
- EFLAGS: A register where each individual bit acts as a status or control flag detailing the outcome of the last ALU operation.

EFLAGS: Worked Binary Arithmetic Examples

Carry Flag (CF)

> **Definition**
>
> Triggers on *unsigned* arithmetic that is out of range — a carry out of the most significant bit on addition, or a borrow required on subtraction.

CF = 1 (carry out occurs)

$$1001\ 0111 \quad + 1010\ 1111 \quad \overline{1\ 0100\ 0110}$$

A 9th bit is produced, exceeding the 8-bit limit. CF = 1.

CF = 0 (no carry out)

$$1001\ 0111 \quad + 0010\ 1111 \quad \overline{1100\ 0110}$$

The result fits entirely within 8 bits — no 9th bit is generated. CF = 0.

Overflow Flag (OF)

Definition

Triggers when a *signed* result falls outside the valid range for the destination size.

Bit Width	Valid Signed Range
4-bit	−8 to 7
8-bit	−128 to 127
16-bit	−32768 to 32767
32-bit	−2147483648 to 2147483647

4-bit Example: $1111\ (-1) - 0101\ (5)$

Compute via 2's complement addition. 2's complement of the subtrahend: $0101 \quad 1011$ (−5).

$$1111\ (-1) \quad + 1011\ (-5) \quad \overline{1\ 1010\ (-6)}$$

Discard the carry out. Result: `1010` (−6).

8-bit Example: $1111\ 0110\ (-10) + 0101\ 1100\ (92)$

$$1111\ 0110\ (-10) \quad + 0101\ 1100\ (92) \quad \overline{1\ 0101\ 0010\ (82)}$$

Discard the carry out. Result: `0101 0010` (82).

Carry-in/carry-out check: the carry *into* the MSB and the carry *out of* the MSB are both 1. $1 \oplus 1 = 0$, so OF = 0. This matches the range check: 82 is well within −128 to 127.

Sign Flag (SF)

Definition

Set to 1 if the Most Significant Bit (MSB) of the result is 1 (the result is negative). Set to 0 if the MSB is 0.

Worked Example: $6 - 13$

Step 1 — Represent 13 and find its negative:

$$13 = 0000\ 1101 \quad 1\text{'s complement} = 1111\ 0010 \quad 2\text{'s complement} = 1111\ 0011 \quad -13$$

Step 2 — Add 6 and –13:

$$\begin{array}{ll} & 0000\ 0110\ (6) \\ + & 1111\ 0011\ (-13) \\ \hline & 1111\ 1001\ (-7) \end{array}$$

Step 3 — Verify the magnitude:

$$1\text{'s complement of } 1111\ 1001 = 0000\ 0110 \quad 2\text{'s complement} = 0000\ 0111 = 7 \quad -7$$

The MSB of the result is `1`, so SF = 1.

Zero Flag (ZF)

Definition

Active only if the result is *completely* zero.

ZF = 0 (OR operation)

$$\begin{array}{ll} & 1111\ 0111 \\ \text{OR} & 0010\ 0001 \\ \hline & 1111\ 0111 \end{array}$$

The result is not all zero. ZF = 0.

ZF = 1 (AND operation)

$$\begin{array}{ll} & 0101\ 0110 \\ \text{AND} & 1010\ 1001 \\ \hline & 0000\ 0000 \end{array}$$

The entire result is zero. ZF = 1.

Auxiliary Carry Flag (AC)

Definition

Triggers when addition produces a carry that moves from bit 3 into bit 4 — that is, from the lower nibble into the upper nibble.

AC = 1

$$\begin{array}{r} 0101\ 1110 \\ +\ 1010\ 1101 \\ \hline 1\ 0000\ 1011 \end{array}$$

The lower nibble (`1110 + 1101`) produces a carry into the upper nibble. AC = 1.

AC = 0

$$\begin{array}{r} 0101\ 1110 \\ +\ 1010\ 0001 \\ \hline 1111\ 1111 \end{array}$$

The lower nibble (`1110 + 0001 = 1111`) does *not* overflow into the upper nibble. AC = 0.

Parity Flag (PF)

Definition

Triggers when the number of `1` bits in the lowest byte of the result is even.

PF = 1

$$\begin{array}{r} 0101\ 1110 \\ +\ 1010\ 0001 \\ \hline 1111\ 1111 \end{array}$$

The result contains exactly eight `1` bits. Since 8 is even, PF = 1.

Summary Table

Flag	Trigger Condition	Example Result
CF	Unsigned result out of range	`1001 0111 + 1010 1111` → CF = 1
OF	Signed result out of range	`1111 0110 + 0101 1100` → OF = 0
SF	MSB of result is 1	`6 − 13 = −7` → SF = 1
ZF	Result is exactly zero	`0101 0110 AND 1010 1001` → ZF = 1
AC	Carry from bit 3 to bit 4	`0101 1110 + 1010 1101` → AC = 1
PF	Even number of 1s in low byte	`0101 1110 + 1010 0001` → PF = 1

Protected Mode Architecture

Protected Mode is the native operating state for modern OS environments like Linux and MS-Windows, designed explicitly for robust multitasking and system stability.

- Memory Capabilities: Grants access to 4 GB of addressable RAM using 32-bit addresses spanning from `00000000h` to `FFFFFFFFh`.
- Partition Security: Every running program is assigned its own memory partition. The processor hardware actively protects these partitions, preventing programs from interfering with or crashing each other.
- Segment Descriptor Tables: Memory program structures (code, data, and stack areas) are managed using segment descriptors stored in tables — primarily the Global Descriptor Table (GDT). Segment registers hold *selectors* that index into this table, not the descriptors themselves.
- Flat Memory Model: To simplify memory management for programmers, modern MASM (Microsoft Macro Assembler) applications use the Microsoft flat memory model, which overlays code, data, and stack segments into a single, continuous 32-bit address space.

Lecture 3 (Activity)

x86 Architecture: Review Questions

Overview

This note is a question-and-answer review of the IA-32 processor: operating modes, registers, **EFLAGS** status flags, and the Floating-Point Unit.

Q1. Basic Modes of Operation

What are the x86 processor's three basic modes of operation?

The three basic modes are:

1. Protected Mode
2. Real-Address Mode
3. System Management Mode

Virtual-8086 Mode

Virtual-8086 Mode is a special case of protected mode, which is why it is not counted as a separate basic mode in this answer. Some texts list it as a fourth mode, so both counts are valid depending on the source.

Q2. The Eight 32-bit General-Purpose Registers

Name all eight 32-bit general-purpose registers.

Register	Full Name
EAX	Extended Accumulator Register
EBX	Extended Base Register
ECX	Extended Counter Register
EDX	Extended Data Register

Register	Full Name
ESI	Extended Source Index
EDI	Extended Destination Index
EBP	Extended Base Pointer
ESP	Extended Stack Pointer

Counting Rule

ESI, EDI, EBP, and ESP are grouped as index and pointer registers in earlier notes, but they still count among the eight general-purpose registers.

Q3. The Six Segment Registers

Name all six segment registers.

Register	Full Name
CS	Code Segment
DS	Data Segment
SS	Stack Segment
ES	Extra Segment
FS	Additional data segment
GS	Additional data segment

Q4. Special Purpose of ECX

What special purpose does the ECX register serve?

ECX (Extended Counter Register) is typically used by the processor as the default loop counter for repetitive operations.

Q5. Stack Variable Registers

Besides ESP, what other register points to variables on the stack?

EBP (Extended Base Pointer) is generally used to reference variables and parameters located on the stack. Because ESP moves as data is pushed and popped, EBP provides a stable reference point.

Q6. CPU Status Flags

Name at least four CPU status flags.

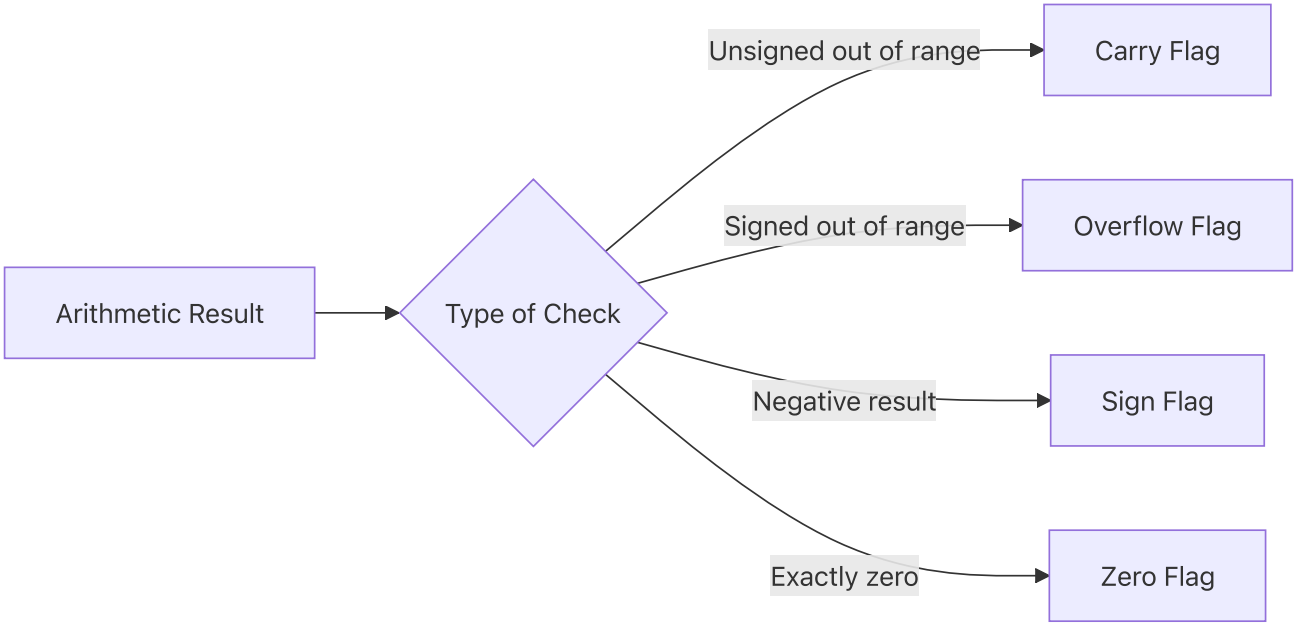
Flag	Name
CF	Carry Flag
ZF	Zero Flag
SF	Sign Flag
OF	Overflow Flag

Q7 to Q9. Flag Identification

Question	Flag	Explanation
Q7. Which flag is set when an *unsigned* result is too large for the destination?	Carry Flag (CF)	An unsigned operation produces a result that exceeds the destination's capacity.
Q8. Which flag is set when a *signed* result is too large or too small for the destination?	Overflow Flag (OF)	A signed calculation falls outside the valid range for the destination operand.
Q9. Which flag is set when an operation generates a *negative* result?	Sign Flag (SF)	The most significant bit of the result is `1`.

Memory Aid

- Carry is for unsigned out-of-range.
- Overflow is for signed out-of-range.
- Sign reports whether the result is negative.



Q10. Floating-Point Arithmetic

Which part of the CPU performs floating-point arithmetic?

The Floating-Point Unit (FPU), a dedicated section of the CPU, handles floating-point arithmetic operations.

Lecture 4

Assembly Language Basics: Constants and Data Types

Overview

This note covers the building blocks of MASM (Microsoft Macro Assembler) programs: integer and string constants, data type attributes, program vocabulary (identifiers, directives, instructions), and a worked hexadecimal arithmetic example.

Integer Constants

Syntax Format

```
[{+|-}] digits [radix]
```

Radix (Base) Identifiers

The base of a number is defined using a trailing character:

Suffix	Base
`d` (or none)	Decimal
`h`	Hexadecimal
`b`	Binary
`q` or `o`	Octal

Examples

- `42` or `30d` → decimal
- `1101b` → binary
- `63q` or `63o` → octal

⚠ Hexadecimal Rule

If a hexadecimal constant begins with a letter (A through F), it must be prefixed with a leading zero so the assembler recognizes it as a number rather than a variable name.

Correct: `0A5h`
Incorrect: `A5h`

Character and String Constants

Characters

Enclosed in single (`'A'`) or double (`"A"`) quotes. Each individual character occupies exactly 1 byte (8 bits) of memory.

ASCII References

- `'A'` = 65
- `'B'` = 66
- `'a'` = 97
- `'b'` = 98

For example, the character `'B'` (66) is stored in binary as:

$$0100\ 0010$$

Strings

Enclosed in single or double quotes (e.g., `"Hello"` or `'Hello'`). Each character within the string takes up a single byte.

Embedded Quotes

Quotes can be included within a string by alternating the quote types:

`'Say "Goodnight," Gracie'`

Data Types (Attributes)

Attributes tell the MASM assembler the size and usage of variables.

Unsigned Types

Type	Size
BYTE	8 bits

Type	Size
`WORD`	16 bits
`DWORD`	32 bits
`QWORD`	64 bits

Signed Types

Prefixed with `S`: `SBYTE`, `SWORD`, `SDWORD`. The Most Significant Bit acts as the sign flag — `1` means negative, `0` means positive.

Program Vocabulary and Structure

Identifiers and Reserved Words

- Identifiers: Programmer-chosen names used for variables, functions, and constants.
- Reserved Words: Words with special built-in meaning in MASM — instruction mnemonics, register names, attributes, operators, predefined symbols (like `@data`), and directives.

Operators

Used to evaluate constant expressions at assembly time, in the following precedence order:

1. Parentheses `()`
2. Unary plus/minus `+`, `-`
3. Multiply `*`, Divide `/`, Modulus `MOD`
4. Add `+`, Subtract `-`

Directives vs. Instructions

Directives

Setup commands that tell the assembler how to organize the program (e.g., `.stack`, `.data`, `.code`). These are not part of the Intel processor's instruction set, and different assemblers use different directives — MASM directives differ from NASM directives.

Instructions

Intel IA-32 commands that are actually converted into machine code and executed by the CPU at runtime. An instruction statement contains up to four parts: Label, Mnemonic, Operand, and Comment.

Anatomy of an Instruction

Labels

Act as place markers for memory addresses.

Label Type	Rule	Example
Data Labels	Must be unique; no colon	`myArray BYTE 58d`
Code Labels	Used as jump/loop targets; must end with a colon	`L1:`

Mnemonics

Action words like `MOV`, `ADD`, `SUB`, `MUL`, `DIV`, `INC` (increment), and `DEC` (decrement).

Operands

The targets of the instruction — constants, expressions, registers, or memory locations.

> Operand Counts
>
> - Zero operands: `stc`
> - One operand: `inc eax`
> - Two operands: `add ebx, ecx`

Comments

- Single-line: start with a semicolon (`;`).
- Multi-line: begin with the `COMMENT` directive followed by a user-chosen symbol, ending when that same symbol appears again.

Code Example and Hexadecimal Arithmetic

MASM programs are structured into sections using directives. Variables are declared in `.data`, and logic goes in `.code` inside a procedure (like `main PROC`).

```
.code
main PROC
    mov ax, 1234h    ; move 1234h into register ax
    mov dx, 5678h    ; move 5678h into register dx

    ; Syntax for two operands is usually: Mnemonic Destination, Source
    add ax, dx       ; Destination (ax) = ax + dx
main endp
end main
```

Arithmetic Breakdown (`ax = ax + dx`)

The processor adds the binary equivalents of the hexadecimal numbers `1234h` and `5678h`.

$$\begin{array}{r} 0001\ 0010\ 0011\ 0100\ (1234)_{\text{h}} \\ +\ 0101\ 0110\ 0111\ 1000\ (5678)_{\text{h}} \\ \hline 0110\ 1000\ 1010\ 1100\ (68A\text{C})_{\text{h}} \end{array}$$

Result

The register `ax` now holds the value `68ACh`.

Lecture 5

Assembly Language Syntax and Program Structure

> **Overview**
>
> This note covers Comments, Labels, Mnemonics and Operands, instruction formats, a worked Add and Subtract program using the Irvine32 library, and suggested coding standards for consistent assembly style.

Comments

Single-Line Comments

Begin with a semicolon (`;`).

Multi-Line Comments

Use the `COMMENT` directive followed by a programmer-chosen delimiter character. The comment block ends when that same delimiter reappears.

> **Using `\` as the Delimiter**

```
COMMENT \
This is the creation of class
This line tells the programmer how to initialize a variable
\
```

Labels

Labels act as markers in a program and can be created in both the `.data` and `.code` segments.

> **⚠ Identifier Rule**
>
> Label names cannot contain spaces. Use `my_Array`, never `my Array`.

Label Type	Segment	Trailing Colon	Purpose
Data Labels	`.data`	No	Declare or initialize variables in memory (e.g., `myArray`).
Code Labels	`.code`	Yes	Serve as jump targets (e.g., `L1:`).

Code Labels and Jumps

Code labels function similarly to the `goto` statement in C++. To perform an unconditional jump to a label, use:

```
jmp L1
```

Mnemonics and Operands

Mnemonics

The action words (keywords) of an instruction, such as `MOV` (used for assigning values), `ADD`, `SUB`, `MUL`, `INC`, and `DEC`.

Operands

The specific targets the mnemonic acts upon. For example, in `mov ah, 32d`, both `ah` and `32d` are operands.

Operand Categories

1. Immediate: A constant value coded directly into the instruction (e.g., `32d`, `5`, `2`).
2. Memory: A data label representing a variable stored in memory (e.g., `myByte`).
3. Registers: A CPU register used directly for holding or assigning values (e.g., `ah`).

Instruction Format Examples

No Operand

`stc` — automatically sets the Carry Flag, so `CF = 1`.

One Operand

- `inc eax` — using a register.
- `inc myByte` — using a memory variable.

Two Operands

- `add ebx, ecx` — register, register.
- `sub myByte, 25` — memory, constant.
- `add eax, 36*25` — register, constant expression.

Code Example: Add and Subtract Program

This program demonstrates adding and subtracting 32-bit integers. It uses the Irvine32 library (`INCLUDE Irvine32.inc`) to enable 32-bit functionality.

```
INCLUDE Irvine32.inc

.code
main PROC
    mov eax, 10000h      ; EAX = 10000h
    add eax, 40000h      ; EAX = 10000h + 40000h = 50000h
    sub eax, 20000h      ; EAX = 50000h - 20000h = 30000h

    call DumpRegs        ; Displays the current state of the Registers
    exit
main ENDP
END main
```

Register Trace

Step	Instruction	EAX Value
1	`mov eax, 10000h`	`10000h`
2	`add eax, 40000h`	`50000h`
3	`sub eax, 20000h`	`30000h`

Suggested Coding Standards

Capitalization Styles

Consistency is key when writing assembly code. Pick one approach and stick to it:

1. Capitalize nothing.
2. Capitalize everything.
3. Capitalize all reserved words, including instruction mnemonics and registers.
4. Capitalize only directives and operators.

Indentation

Ensure clear spacing and indentation between distinct program segments, such as `.data` and `.code`, to maintain readability.

Assembling, Linking, and Running Programs

Assembling, Linking, and Running Programs

Overview

This note covers the Assemble-Link-Execute Cycle, the `make32.bat` build automation script, and the two key system output files — the Listing File (`.lst`) and the Map File (`.map`) — produced during the build process.

Exam Note

This material was flagged as highly important theory for exams, despite being covered in the same lecture as earlier topics.

The Assemble-Link-Execute Cycle

This cycle describes the exact sequential steps required to go from writing a source program to executing the compiled program.

Important Repetition Rule

Whenever the source code is modified, Steps 2 through 4 must be repeated to apply the changes. Step 1 only needs to happen once per edit.



Step	Tool	Input	Output
1	Text Editor	—	Source File
2	Assembler	Source File	Object File, Listing File
3	Linker	Object File + Link Library	Executable File, Map File
4	OS Loader	Executable File	Final Output

Build Automation: `make32.bat`

Instead of running the assembler and linker manually, developers use batch files to automate the build process.

What It Does

`make32.bat` is a batch file used to assemble and link 32-bit programs. It contains the automated commands to execute:

- `ML.EXE` — the Microsoft Assembler
- `LINK32.EXE` — the 32-bit Microsoft Linker

Command-Line Syntax

```
make32 progName
```

The `progName` must not include the `.asm` file extension.

⚠ Real-Mode Alternative

To assemble and link older Real-mode programs, use `make16.bat` instead.

System Output Files

During the build cycle, the assembler and linker produce two important reference files used for debugging and memory tracking.

1. The Listing File (`.lst`)

Purpose

Allows the programmer to look under the hood and see exactly how the program was compiled.

Example file name: `addSub.lst`

It contains a comprehensive breakdown of the build, including:

- The original source code
- Memory addresses
- The generated object code (raw machine language)
- Segment names

- Symbols (variables, procedures, and constants)

2. The Map File (`.map`)

> **Purpose**
>
> Provides specific structural information about each individual program segment.

Example file name: `addSub.map`

The details provided for each segment include:

Detail	Description
Starting Address	Where the segment begins in memory.
Ending Address	Where the segment ends in memory.
Segment Size	The total size of the segment.
Segment Type	The classification of the segment (e.g., code, data).

Quick Reference Comparison

File	Extension	Produced By	Primary Use
Listing File	`.lst`	Assembler	Source-to-machine-code breakdown, debugging
Map File	`.map`	Linker	Segment addresses and sizes, memory layout

Lecture 6

Assembly Data Types and Memory Storage

> **Overview**
>
> This note covers IEEE floating-point data types, Data Definition Statements for bytes, words, and double words, string and null-termination conventions, the DUP Operator for array allocation, and Little Endian memory ordering on Intel processors.

Intrinsic Data Types (Floating Point)

In addition to standard integer types, Assembly supports specific IEEE Floating-Point types:

Type	Size	Description
REAL4	4 bytes	IEEE short real
REAL8	8 bytes	IEEE long real
REAL10	10 bytes	IEEE extended real

Data Definition Statements

A data definition statement sets aside memory storage for a variable and optionally assigns a name (or label) to that data.

> **Syntax**
>
> `[name] directive initializer [,initializer] . . .`

- All initializers are ultimately converted into binary data in memory.
- When a variable name (like `value1`) is assigned, it acts as a data label that implies a specific memory offset (an address) for that data.

Syntax Breakdown

Component	Required?	Meaning
`[name]`	Optional	The variable name (e.g., `count`, `myString`). If omitted, the assembler still allocates the memory — there is simply no convenient name to reference it by later.
`directive`	Required	Defines the size of the memory chunk to carve out (e.g., `BYTE` carves out 8 bits, `DWORD` carves out 32 bits).
`initializer`	Required	The starting value placed inside that carved-out chunk. Using `?` tells the assembler to reserve the space but leave it uninitialized.

Conversion to Binary

The CPU does not understand decimal numbers, hex codes, or text characters directly. Whether `65` (decimal), `41h` (hex), or `'A'` (character) is used as the initializer, the assembler translates all of them into the exact same raw 8-bit binary pattern before writing it to the hardware:

$$65_{10} = 41_{16} = \text{'A'} = 0100\ 0001_2$$

The Reality of "Variables" (Memory Offsets)

In high-level languages (like C++ or Java), a variable feels like a discrete container. In Assembly, there are no true variables. A name like `value1` is strictly a data label — a human-readable alias for a raw numeric memory address (an offset).

- The assembler tracks how many bytes each piece of data occupies. If `value1` is the first byte in the data segment, its offset is `0000`. If `value2` comes right after it, its offset is `0001`.
- When writing `mov al, value1`, the assembler simply substitutes the word `value1` with its raw hardware address. The label just saves the programmer from manually calculating and memorizing numeric addresses.

Defining Bytes (8-bit)

You can define single bytes using the `BYTE` (unsigned) or `SBYTE` (signed) directives.

- Constants: Can be character constants like `'A'`.
- Unsigned Limits: From `0` to `255`.
- Signed Limits: From `-128` to `+127`.

- Uninitialized: Use a question mark (`?`) to leave a byte uninitialized.
- Multiple Initializers: You can declare multiple bytes sequentially by separating them with commas (e.g., `list1 BYTE 10,20,30,40` or `list4 BYTE 0Ah, 20h, 'A', 22h`).
- Breaking Lists Across Multiple Lines: When defining an array with a large number of initializers, you can break the list across multiple lines to improve readability. Declare the variable name on the first line, then simply repeat the data type directive on subsequent lines without retyping the variable name.

Multi-Line Byte List

```
list2 BYTE 10, 20, 30, 40
      BYTE 50, 60, 70, 80
      BYTE 81, 82, 83, 84
```

Defining Words (16-bit)

Use `WORD` (unsigned) or `SWORD` (signed) for 16-bit storage. Words can store 16-bit integers or double characters (e.g., `"AB"`).

Property	Value
Largest Unsigned Value	`65535`
Smallest Signed Value	`-32768`

Defining Double Words (32-bit)

Use `DWORD` (unsigned) or `SDWORD` (signed) to allocate 32-bit integers.

Declarations

- `val1 DWORD 12345678h` — unsigned 32-bit value.
- `val2 SDWORD -2147483648` — signed 32-bit value.

Defining Strings

A string is implemented as an array of characters, usually enclosed in quotation marks.

Null Termination

Strings usually have a null byte (`0`) at the end to mark where the string stops:

```
str1 BYTE "Enter your name",0
```

Multi-line Strings

To continue a single string across multiple lines without breaking it, end each line with a comma.

End-of-Line Characters

Hex Code	Meaning
`0Dh`	Carriage Return
`0Ah`	Line Feed

Combined Newline Declaration

```
newLine BYTE 0Dh, 0Ah, 0
```

Array Allocation: The DUP Operator

The DUP Operator is used to easily allocate space for an array or a repeating string.

⚠ Constraint

Both the counter and the argument must be constants or constant expressions.

Example 1 — Zero-Initialized Array

`var1 BYTE 20 DUP(0)` creates 20 bytes, all initialized to zero.

Example 2 — Uninitialized Array

`var2 BYTE 20 DUP(?)` creates 20 uninitialized bytes.

Example 3 — Repeating String

`var3 BYTE 4 DUP("STACK")` creates a 20-byte string:

`"STACKSTACKSTACKSTACK"`

(4 repetitions × 5 characters = 20 bytes)

Memory Architecture: Little Endian Order

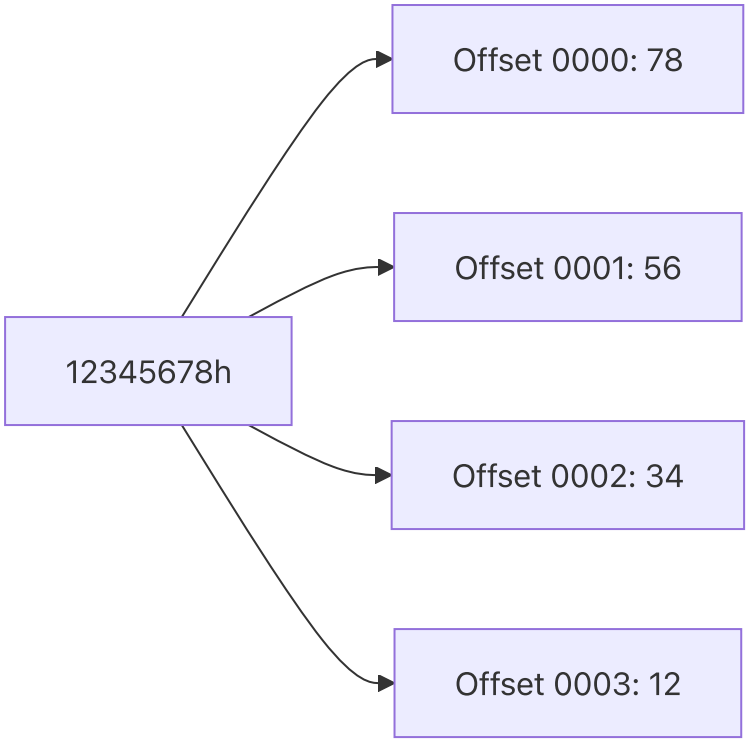
Intel processors use Little Endian order for storing data in memory.

Rule

- All data types larger than a single byte store their individual bytes in reverse order.
- The least significant byte is placed at the first (lowest) memory address.

Breakdown: `val1 DWORD 12345678h`

Offset	Byte
0000	78
0001	56
0002	34
0003	12


12345678h
Offset 0000: 78
Offset 0001: 56
Offset 0002: 34
Offset 0003: 12

Lecture 7

Variables, Symbolic Constants, and Real-Address Mode

Overview

This note covers a MASM program that does arithmetic with memory variables, the `.data?` directive for uninitialized data, the three symbolic constant directives (`=` , `EQU` , `TEXTEQU`), automatic array size calculation with the Location Counter (`$`), and how to convert a 32-bit program into a 16-bit Real-Address Mode program.

---

1. Adding and Subtracting with Variables (`AddSub2.asm`)

To do arithmetic with memory variables instead of hardcoded numbers, declare the variables in the `.data` segment and manipulate them through registers in the `.code` segment.

Complete Program

```
TITLE Add and Subtract, Version 2 (AddSub2.asm)
; This program adds and subtracts 32-bit unsigned
; integers and stores the sum in a variable.
INCLUDE Irvine32.inc

.data
val1     DWORD 10000h
val2     DWORD 40000h
val3     DWORD 20000h
finalVal DWORD ?

.code
main PROC
    mov eax, val1      ; start with 10000h
    add eax, val2      ; add 40000h
    sub eax, val3      ; subtract 20000h
    mov finalVal, eax  ; store the result (30000h)
    call DumpRegs      ; display the registers
    exit
main ENDP
END main
```

Header and Setup

TITLE and INCLUDE

- `TITLE` sets a descriptive title that appears in the Listing File header. It does not name or save the file.
- `INCLUDE Irvine32.inc` brings in the Irvine32 library definitions, including `DumpRegs` and `exit`.

Variable Declarations (`.data`)

Variable	Type	Initial Value
`val1`	`DWORD`	`10000h`
`val2`	`DWORD`	`40000h`
`val3`	`DWORD`	`20000h`
`finalVal`	`DWORD`	`?` (uninitialized, holds the answer later)

Execution Steps (`.code`)

Step	Instruction	Effect	EAX After
1	`mov eax, val1`	Copies `val1` from memory into `eax`.	`10000h`
2	`add eax, val2`	Adds `40000h`.	`50000h`
3	`sub eax, val3`	Subtracts `20000h`.	`30000h`
4	`mov finalVal, eax`	Copies the result into memory.	`30000h`
5	`call DumpRegs`	Displays the CPU register state.	`30000h`

$$10000h + 40000h - 20000h = 30000h$$

Why Two Steps to Store

`mov finalVal, eax` is needed because the arithmetic happens in the register. The variable in memory is only updated when the result is copied back.

---

2. Declaring Uninitialized Data (`.data?`)

For large uninitialized variables or arrays, use the `.data?` directive instead of placing everything in the standard `.data` segment.

How It Works

Use `.data?` exclusively to declare an uninitialized data segment.

Advantage

It reduces the size of the compiled `.EXE` file. Initialized data must be stored inside the file, but uninitialized data only needs to be stored at load time.

Comparison

Inefficient: Mixing Both in `.data`

```
.data
smallArray DWORD 10 DUP(0)
bigArray   DWORD 5000 DUP(?)
```

Efficient: Splitting with `.data?`

```
.data
smallArray DWORD 10 DUP(0)

.data?
bigArray   DWORD 5000 DUP(?)
```

`5000 DUP(?)` duplicates an uninitialized `DWORD` 5,000 times:

$$5000 \times 4 = 20{,}000 \text{ bytes}$$

---

3. Symbolic Constants and Array Size Calculations

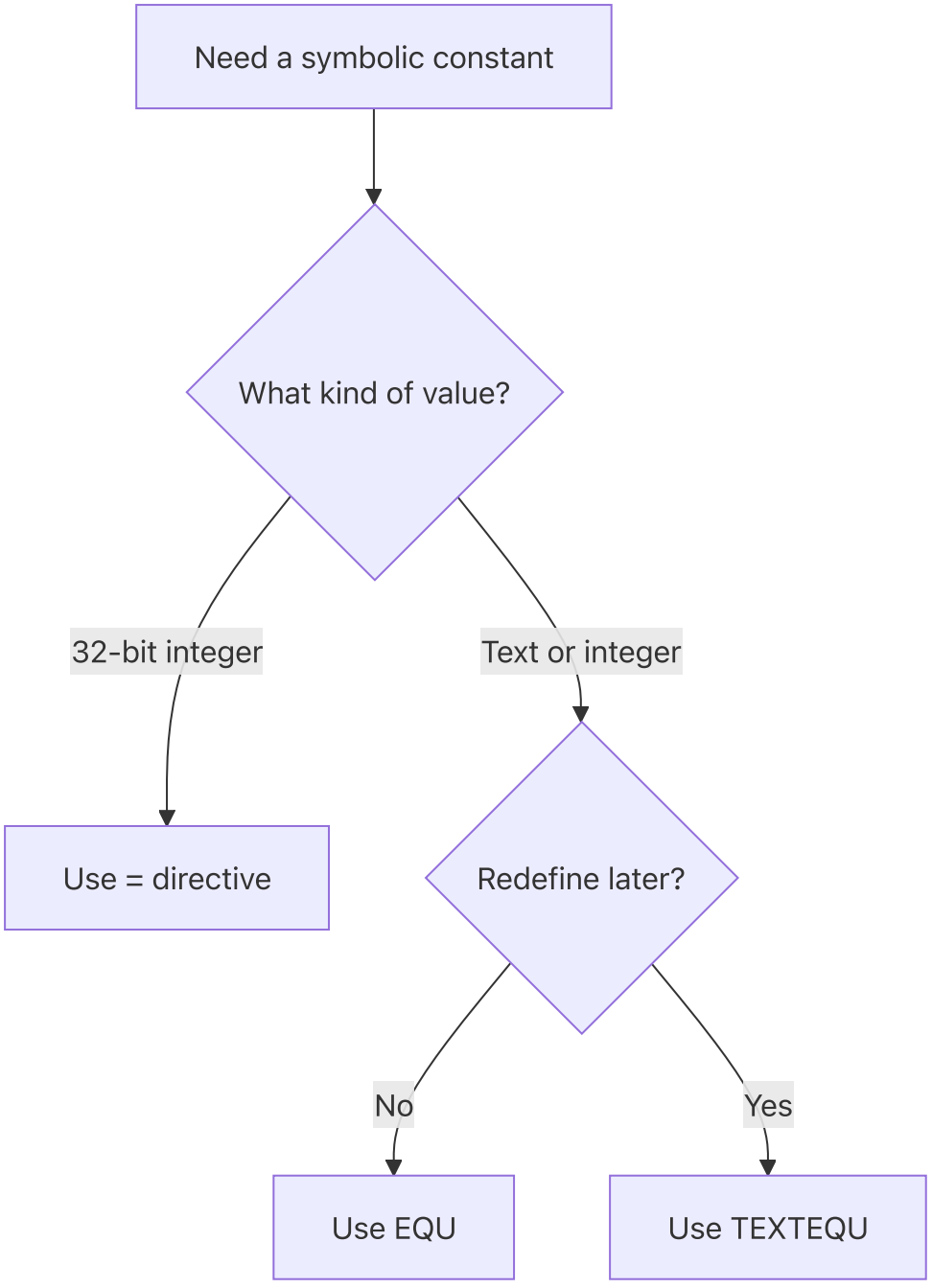
A symbolic constant is a name assigned to a value or text expression.

Key Properties

- Zero storage cost: unlike a variable, a symbolic constant uses no runtime memory.
- Assembly-time only: symbols are evaluated by the assembler during assembly, never at runtime.

The Three Directives Compared

Directive	What It Defines	Can It Be Redefined?
`=` (Equal-Sign)	32-bit integer expression or constant	Yes
`EQU`	Integer or text expression	No
`TEXTEQU`	Integer or text expression (text macro)	Yes



A. Equal-Sign Directive (`=`)

Syntax

```
name = expression
```

- The `expression` must be a 32-bit integer (a math expression or a single constant).
- The `name` is the symbolic constant. Using symbols instead of raw numbers is considered good programming style.
- Unlike `EQU`, a symbol created with `=` may be redefined at any point in the program.

Assigning a Key Code

```
Esc_key = 27
mov al, Esc_key
```

Redefining a Symbol Mid-Program

```
.data
Count = 5
array1 BYTE Count DUP(0)    ; reserves 5 bytes
Count = 10
array2 BYTE Count DUP(0)    ; reserves 10 bytes

.code
mov al, Count               ; AL = 10 (the current value)
```

B. Calculating the Sizes of Arrays and Strings

Instead of counting elements by hand, let the assembler calculate the size using the `=` directive and the Location Counter (`$`).

How the Math Works

Subtracting the starting address of the array (`list`) from the current location counter (`$`) gives the total number of bytes the array occupies. To get the number of elements, divide that byte total by the size of the data type.

⚠ Placement Rule

The `ListSize` line must come **immediately after** the array declaration. If other data is declared in between, `$` has moved on and the result will include that extra data.

Array Type	Bytes per Element	Formula	Elements
`BYTE`	1	`($ - list)`	4
`WORD`	2	`($ - list) / 2`	4
`DWORD`	4	`($ - list) / 4`	4

Byte Array (1 byte per element)

The byte difference directly equals the element count.

```
list BYTE 10,20,30,40
ListSize = ($ - list)
```

$$4 \text{ bytes} \div 1 = 4 \text{ elements}$$

Word Array (2 bytes per element)

Divide the byte total by 2.

```
list WORD 1000h, 2000h, 3000h, 4000h
ListSize = ($ - list) / 2
```

$$8 \text{ bytes} \div 2 = 4 \text{ elements}$$

Doubleword Array (4 bytes per element)

Divide the byte total by 4.

```
list DWORD 1,2,3,4
ListSize = ($ - list) / 4
```

$$16 \text{ bytes} \div 4 = 4 \text{ elements}$$

C. `EQU` Directive

Defines a symbol as either an integer or a text expression. Once defined, an `EQU` symbol cannot be redefined later in the program.

Three Syntax Formats

1. `Name EQU expression` — a valid integer expression.
2. `Name EQU Symbol` — an existing symbol name.
3. `Name EQU <text>` — any text placed inside angle brackets `< >`.

Examples

```
PI EQU <3.1416>
pressKey EQU <"Press any key to continue...",0>

.data
prompt BYTE pressKey
```

D. `TEXTEQU` Directive

Defines a symbol as either an integer or a text expression. A symbol created with `TEXTEQU` is called a Text Macro, and unlike `EQU`, it can be redefined at any time.

Three Syntax Formats

1. `name TEXTEQU <text>` — assigns literal text inside angle brackets.
2. `name TEXTEQU textmacro` — assigns the contents of an existing text macro.
3. `name TEXTEQU %constExpr` — the `%` prefix tells the assembler to evaluate a constant integer expression and convert the result into text.

Examples

```
continueMsg TEXTEQU <"Do you wish to continue (Y/N)?">
.data
prompt1 BYTE continueMsg

rowSize = 5
count   TEXTEQU %(rowSize * 2)  ; Evaluates to 10
```

```
move    TEXTEQU <mov>
setupAL TEXTEQU <mov al, count> ; Expands to: mov al, 10
```

---

4. Real-Address Mode Programming (16-Bit MS-DOS)

Protected Mode produces 32-bit programs. Real-Address Mode is used to generate 16-bit MS-DOS programs.

Advantages and Disadvantages

Advantages	Disadvantages
Enables direct calling of MS-DOS and BIOS hardware functions.	You must manually keep track of both memory segments and offsets.
No memory access restrictions.	Cannot call modern Win32 functions (Windows 95 onward).
	Strictly limited to 640K of program memory.

Code Requirements

To convert a 32-bit Protected Mode program into a 16-bit Real-Address Mode program, make two changes:

1. Include the 16-bit library: use `INCLUDE Irvine16.inc` instead of `Irvine32.inc`.
2. Initialize the Data Segment (`DS`) register: manually point `DS` to the `.data` segment at the very start of `main PROC` using the `@data` symbol.

Initializing DS

```
mov ax, @data
mov ds, ax
```

Why the Extra Step

In real-address mode, the program must tell the CPU where its data segment lives. The segment registers are not set up for you the way the flat model does it in protected mode. `DS` cannot be loaded with an immediate value directly, so the address goes through `ax` first.

Complete 16-Bit Example (`AddSub2.asm` , 16-Bit Version)

```
TITLE Add and Subtract, Version 2 (AddSub2.asm)
INCLUDE Irvine16.inc

.data
val1     DWORD 10000h
val2     DWORD 40000h
val3     DWORD 20000h
finalVal DWORD ?

.code
main PROC
    mov ax, @data      ; initialize DS
    mov ds, ax

    mov eax, val1      ; get first value
    add eax, val2      ; add second value
    sub eax, val3      ; subtract third value
    mov finalVal, eax  ; store the result
    call DumpRegs      ; display registers
    exit
main ENDP
END main
```

Save Under a Different Name

This 16-bit version reuses the title `AddSub2.asm` . If you save it in the same folder as the 32-bit version, it will overwrite it, so give it a distinct file name.

32-Bit vs 16-Bit Comparison

Aspect	Protected Mode (32-bit)	Real-Address Mode (16-bit)
Library	`Irvine32.inc`	`Irvine16.inc`
DS setup	Not needed	`mov ax, @data` then `mov ds, ax`
Build file	`make32.bat`	`make16.bat`
Memory limit	4 GB	640K
Win32 calls	Allowed	Not allowed

Aspect	Protected Mode (32-bit)	Real-Address Mode (16-bit)
MS-DOS and BIOS calls	Restricted	Direct access

Lecture 8

Operands, MOV, and Data Transfer Instructions

> **Overview**
>
> This note covers the three operand types, the shorthand notation used in instruction manuals, Direct Memory Operands, the strict rules of the MOV Instruction, MOVZX and MOVSX for widening values, XCHG, and Direct-Offset Operands for array access. It closes with the solved slide activities.

---

1. Operand Types

An operand is the value, register, or memory location that an instruction works on. There are three basic types.

Type	Description	Examples
Immediate	A constant integer of 8, 16, or 32 bits. Fixed at assembly time, so it is encoded directly inside the machine instruction.	`5`, `100`, `20h`
Register	The name of a CPU register. The assembler converts the name to a binary number and encodes it in the instruction.	`AL`, `BX`, `EAX`
Memory	A reference to a location in system memory. Either the exact address is encoded in the instruction, or a register holds the address.	`var1`, `count`

---

2. Instruction Operand Notation

Manuals use shorthand to show which operand types and sizes an instruction accepts.

Symbol	Meaning
`r8`	8-bit general register: `AH`, `AL`, `BH`, `BL`, `CH`, `CL`, `DH`, `DL`
`r16`	16-bit general register: `AX`, `BX`, `CX`, `DX`, `SI`, `DI`, `SP`, `BP`
`r32`	32-bit general register: `EAX`, `EBX`, `ECX`, `EDX`, `ESI`, `EDI`, `ESP`, `EBP`
`reg`	Any general-purpose register
`sreg`	16-bit segment register: `CS`, `DS`, `SS`, `ES`, `FS`, `GS`

Symbol	Meaning
`imm`	Any 8-, 16-, or 32-bit immediate value
`imm8`	8-bit immediate byte
`imm16`	16-bit immediate word
`imm32`	32-bit immediate doubleword
`r/m8`	8-bit register or memory byte
`r/m16`	16-bit register or memory word
`r/m32`	32-bit register or memory doubleword
`mem`	Any 8-, 16-, or 32-bit memory operand

---

3. Direct Memory Operands

A direct memory operand is a named reference (a variable label) pointing to storage in memory.

> **Automatic Dereferencing**
>
> When a variable name appears in an instruction, the assembler dereferences it. It goes to that memory address and fetches the value stored there.

The name can be written plain (`var1`) or in square brackets (`[var1]`). Both do the same thing in MASM.

```
.data
var1 BYTE 10h

.code
mov al, var1    ; AL = 10h
mov al, [var1]  ; AL = 10h (alternate format)
```

---

4. The MOV Instruction

`MOV` copies data from a source operand to a destination operand.

```
MOV destination, source
```

Strict Rules

⚠ Five Rules for MOV

1. **Matching sizes**: both operands must be the same bit size (8 to 8, 16 to 16, 32 to 32).
2. **At most one memory operand**: memory-to-memory moves are not allowed in a single `MOV`.
3. **Protected registers**: `CS`, `EIP`, and `IP` cannot be the destination. Changing them directly would derail program execution.
4. **No immediate-to-segment moves**: a constant cannot go straight into `DS`, `ES`, `SS`, and so on. Pass it through a general register first.
5. **No immediate destinations**: `mov 25, al` is meaningless because the number 25 cannot be overwritten.

Valid vs. Invalid Examples

```
.data
count BYTE 100
wVal  WORD 2

.code
mov bl, count    ; Valid: both 8-bit
mov ax, wVal     ; Valid: both 16-bit
mov count, al    ; Valid: both 8-bit

mov al, wVal     ; ERROR: wVal is 16-bit, AL is 8-bit
mov ax, count    ; ERROR: count is 8-bit, AX is 16-bit
mov eax, count   ; ERROR: count is 8-bit, EAX is 32-bit
```

🖉 Assembly-Time Errors

The three invalid lines are rejected by the assembler because of operand size mismatches. The program never builds.

Solved Activity: "Your Turn..." (Slide 8)

Task: Explain why each `MOV` statement is invalid.

```
.data
bVal  BYTE  100
bVal2 BYTE  ?
wVal  WORD  2
```

```
dVal  DWORD 5

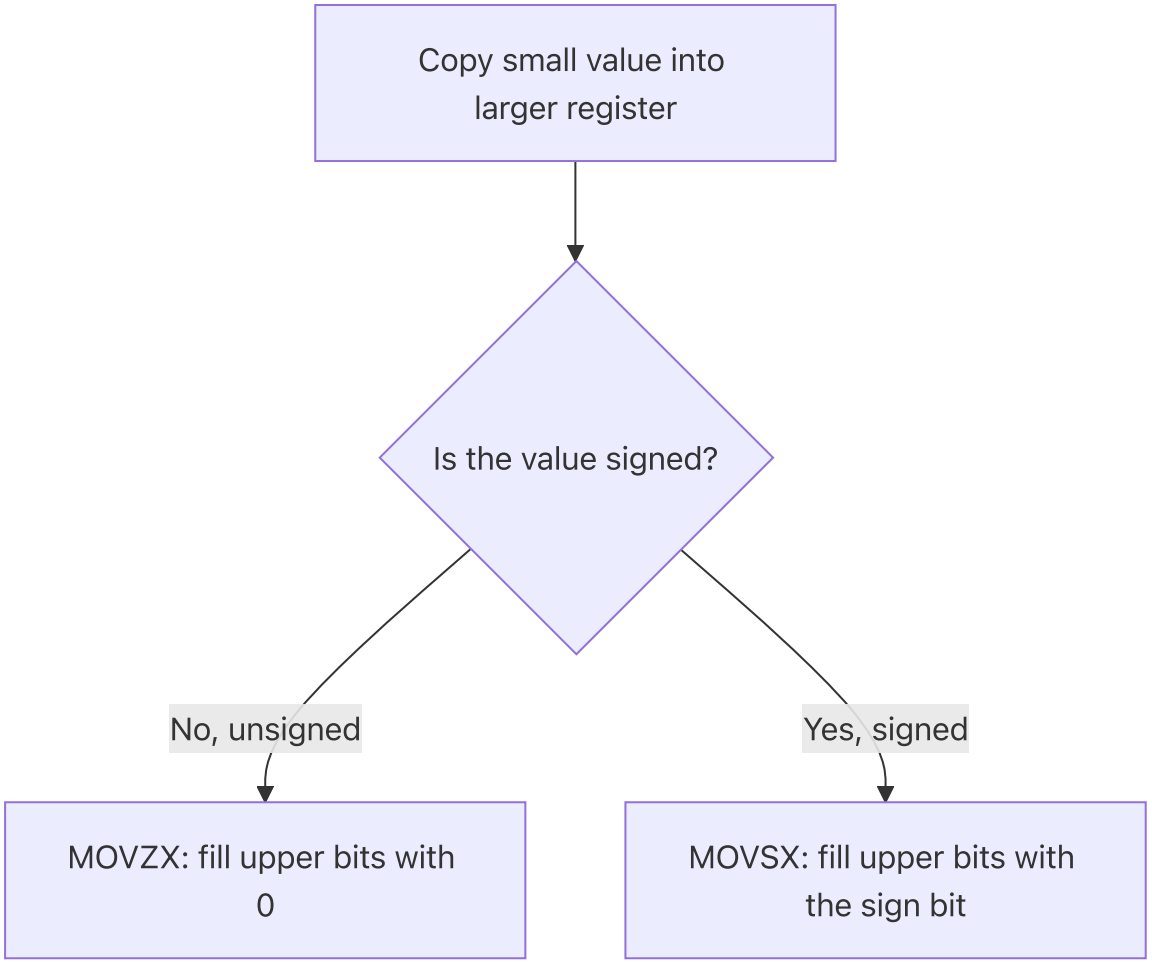
.code
mov ds, 45          ; a.
mov esi, wVal       ; b.
mov eip, dVal       ; c.
mov 25, bVal        ; d.
mov bVal2, bVal     ; e.
```

Part	Statement	Reason Invalid
a.	`mov ds, 45`	**Immediate to segment move.** `DS` is a segment register and `45` is an immediate. A constant cannot be moved directly into a segment register.
b.	`mov esi, wVal`	**Size mismatch.** `ESI` is 32-bit (`r32`) and `wVal` is a 16-bit `WORD` .
c.	`mov eip, dVal`	**Restricted destination.** `EIP` cannot be the destination of a `MOV` .
d.	`mov 25, bVal`	**Immediate destination.** `25` is a constant, not a writable storage location.
e.	`mov bVal2, bVal`	**Memory to memory.** Both operands are memory operands, and only one is allowed.

---

5. Copying Smaller Values into Larger Destinations

A plain `MOV` cannot copy a `BYTE` into a 16-bit or 32-bit register. To widen a value legally, the CPU must be told how to fill the extra upper bits.



> ⚠ **Shared Rule**
>
> For both instructions, the destination must be a register.

Zero Extension: MOVZX (Unsigned)

Fills the upper half of the destination with zeros.

```
mov   bl, 10001111b
movzx ax, bl          ; zero-extension
```

$$\begin{aligned} \text{Source } (BL) &= 1000\ 1111 \\ \text{Destination } (AX) &= 0000\ 0000\ 1000\ 1111 \end{aligned}$$

Sign Extension: MOVSX (Signed)

Filling with zeros would turn a negative number positive by changing its leftmost bit from `1` to `0`. `MOVSX` instead fills the upper half with a copy of the source's sign bit.

```
mov   bl, 10001111b
movsx ax, bl          ; sign-extension
```

$$\begin{aligned} \text{Source } (BL) &= 1000\ 1111 \quad (\text{sign bit} = 1) \\ \text{Destination } (AX) &= 1111\ 1111\ 1000\ 1111 \end{aligned}$$

If the Sign Bit Were 0

If the leftmost bit of `BL` were `0`, `MOVSX` would fill the upper half with `0000 0000`, giving the same result as `MOVZX`.

Instruction	Fills Upper Bits With	Use For
`MOVZX`	`0`	Unsigned values
`MOVSX`	Copy of the sign bit	Signed values

---

6. The XCHG Instruction

`XCHG` swaps the values of two operands.

⚠ Rules for XCHG

1. At least one operand must be a register. Two memory variables cannot be swapped directly.
2. No immediate operands are allowed.
3. Both operands must be the same size.

```
.data
var1 WORD 1000h
var2 WORD 2000h

.code
xchg ax, bx      ; Valid: two 16-bit registers
xchg ah, al      ; Valid: two 8-bit registers
xchg var1, bx    ; Valid: 16-bit memory and 16-bit register
xchg eax, ebx    ; Valid: two 32-bit registers

xchg var1, var2  ; ERROR: two memory operands
```

Swapping Two Memory Variables

Load one variable into a register first, then use `XCHG` between the register and the other variable.

---

7. Direct-Offset Operands

A direct-offset operand accesses an array element by adding a constant byte offset to a variable label.

> **Effective Address (EA)**
>
> Adding a constant to a data label produces an Effective Address, the exact memory address of the target element. That address is then dereferenced to fetch the value stored there.

Byte Arrays (1 byte per step)

```
.data
arrayB BYTE 10h, 20h, 30h, 40h

.code
mov al, arrayB+1    ; AL = 20h
mov al, [arrayB+1]  ; AL = 20h (alternative notation)
```

Expression	Offset	Value
`arrayB`	`+0`	`10h`
`arrayB+1`	`+1`	`20h`
`arrayB+2`	`+2`	`30h`
`arrayB+3`	`+3`	`40h`

> **Why doesn't `arrayB+1` produce `11h`?**
>
> The `+1` is added to the memory address of `arrayB` at assembly time, not to the value `10h` stored inside it. Stepping 1 byte forward from the start lands on the second element, which holds `20h`.

Word and Doubleword Arrays (stepping by byte size)

Memory addresses count bytes, whatever the data type. Offsets must therefore be multiples of the element size.

Array Type	Bytes per Element	Offsets
`WORD`	2	`+0`, `+2`, `+4`, `+6`
`DWORD`	4	`+0`, `+4`, `+8`, `+12`

```
.data
arrayW WORD  1000h, 2000h, 3000h
arrayD DWORD 1, 2, 3, 4

.code
mov ax, [arrayW+2]   ; AX = 2000h (2nd element)
mov ax, [arrayW+4]   ; AX = 3000h (3rd element)
mov eax, [arrayD+4]  ; EAX = 00000002h (2nd element)
```

Will these statements assemble and run?

```
mov ax, [arrayW-2]   ; ??
mov eax, [arrayD+16] ; ??
```

Yes, both assemble and run. MASM does not bounds-check effective addresses. Both are still out-of-bounds bugs:

1. `[arrayW-2]` reads 2 bytes before `arrayW`, loading unrelated data into `AX`.
2. `arrayD` has four elements at offsets `+0`, `+4`, `+8`, and `+12`. `[arrayD+16]` reads 4 bytes past the end, pulling unrelated memory into `EAX`.

---

8. Solved Activities

Activity 1: Rearranging a Doubleword Array

Task: Rearrange `arrayD DWORD 1, 2, 3` so the array holds `3, 1, 2`.

Position	Address	Initial Value
1st	`arrayD` (`+0`)	`1`
2nd	`[arrayD+4]` (`+4`)	`2`
3rd	`[arrayD+8]` (`+8`)	`3`

```
.data
arrayD DWORD 1, 2, 3

.code
mov  eax, arrayD        ; EAX = 1
xchg eax, [arrayD+4]    ; EAX = 2, array = 1, 1, 3
xchg eax, [arrayD+8]    ; EAX = 3, array = 1, 1, 2
mov  arrayD, eax        ; array = 3, 1, 2
```

Step	Instruction	EAX	Array State
1	`mov eax, arrayD`	`1`	`1, 2, 3`
2	`xchg eax, [arrayD+4]`	`2`	`1, 1, 3`
3	`xchg eax, [arrayD+8]`	`3`	`1, 1, 2`
4	`mov arrayD, eax`	`3`	`3, 1, 2`

Activity 2: "Evaluate This..." Adding Three Bytes

Task: Sum the three bytes below.

```
.data
myBytes BYTE 80h, 66h, 0A5h
```

The true sum is:

$$80h + 66h + A5h = 128 + 102 + 165 = 395 = 18Bh$$

> ⚠ **Why This Is a Trap**
>
> `18Bh` is larger than `FFh` (255), so the result needs more than 8 bits.

Part 1: First Snippet

```
mov al, myBytes
add al, [myBytes+1]
add al, [myBytes+2]
```

This assembles and runs but gives the wrong answer because of 8-bit unsigned overflow.

Step	Instruction	AL	CF	OF
1	`mov al, myBytes`	`80h`	unchanged	unchanged
2	`add al, [myBytes+1]`	`E6h`	0	0
3	`add al, [myBytes+2]`	`8Bh`	1	0

$$\begin{array}{r} 1110\ 0110\ (E6h) \\ +\ 1010\ 0101\ (A5h) \\ \hline 1\ 1000\ 1011\ (18Bh) \end{array}$$

The carry bit is lost, so only `8Bh` remains in `AL`.

> ⚠ **Correction to the Source**
>
> The source claimed `OF = 1` at step 2. As signed values, $-128 + 102 = -26$, which is inside the range of -128 to 127, so **OF = 0**. Step 3 is also OF = 0, since $-26 + (-91) = -117$ is in range. Only CF changes.

Part 2: Second Snippet

```
mov ax, myBytes
add ax, [myBytes+1]
add ax, [myBytes+2]
```

This **does not assemble**. `AX` is 16-bit but `myBytes` is an 8-bit `BYTE`, so every line is a size mismatch.

Part 3: The Proposed Fix

```
movzx ax, myBytes
mov   bl, [myBytes+1]
add   ax, bx
mov   bl, [myBytes+2]
add   ax, bx          ; AX = sum
```

> ⍰ **Is anything missing?**
>
> Yes: **`mov bx, 0` must run before the first `add`**. The code loads only `BL`, but `add ax, bx` adds the **whole** 16-bit `BX`. Whatever garbage sits in `BH` is added into the sum.

```
mov   bx, 0           ; clear BH and BL
movzx ax, myBytes
mov   bl, [myBytes+1]
add   ax, bx
mov   bl, [myBytes+2]
add   ax, bx          ; AX = 018Bh
```

Step	Instruction	AX	BX
1	`mov bx, 0`	unchanged	`0000h`
2	`movzx ax, myBytes`	`0080h`	`0000h`
3	`mov bl, [myBytes+1]`	`0080h`	`0066h`
4	`add ax, bx`	`00E6h`	`0066h`
5	`mov bl, [myBytes+2]`	`00E6h`	`00A5h`
6	`add ax, bx`	`018Bh`	`00A5h`

Alternative Fix

Replace each `mov bl, ...` with `movzx bx, [myBytes+n]`. `MOVZX` zeroes `BH` automatically, so `mov bx, 0` is unnecessary.

Quick Reference

Instruction	Purpose	Key Restriction
`MOV`	Copy source to destination	Same size, at most one memory operand
`MOVZX`	Widen an unsigned value	Destination must be a register
`MOVSX`	Widen a signed value	Destination must be a register
`XCHG`	Swap two operands	At least one register, no immediates